Unit 5

Multithreaded Programming: Introduction, Need for Multiple Threads Multithreaded Programming for Multi-core Processor, Thread Class, Main Thread-Creation of New Threads, Thread States, Thread Priority

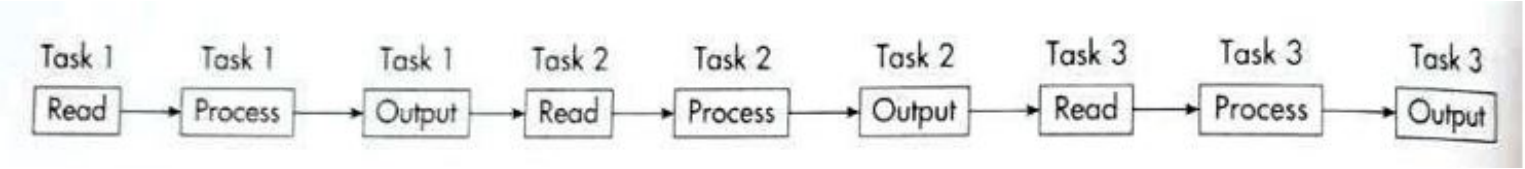
Collections Framework : Introduction, Purpose of Collection Framework Application of Collection Framework, Hierarchy of collection Interfaces / classes, Methods defined in Collection Interface, Interface Iterator, Collection classes/Interfaces –List,Queue,Set, ArrayList class, HashSet, PriorityQueue

Threads

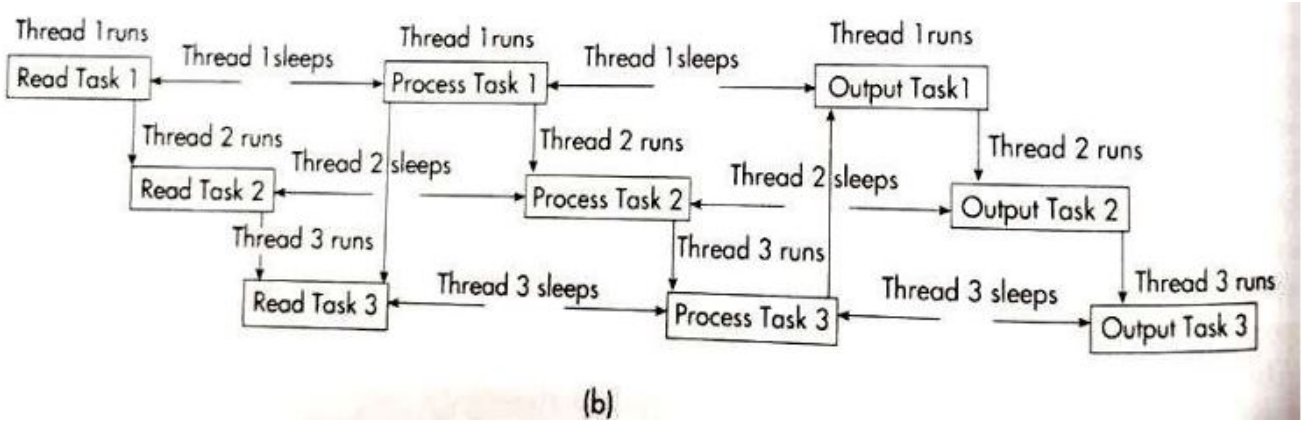
- Single Tasking
  - Single task at a time



Single Processor:

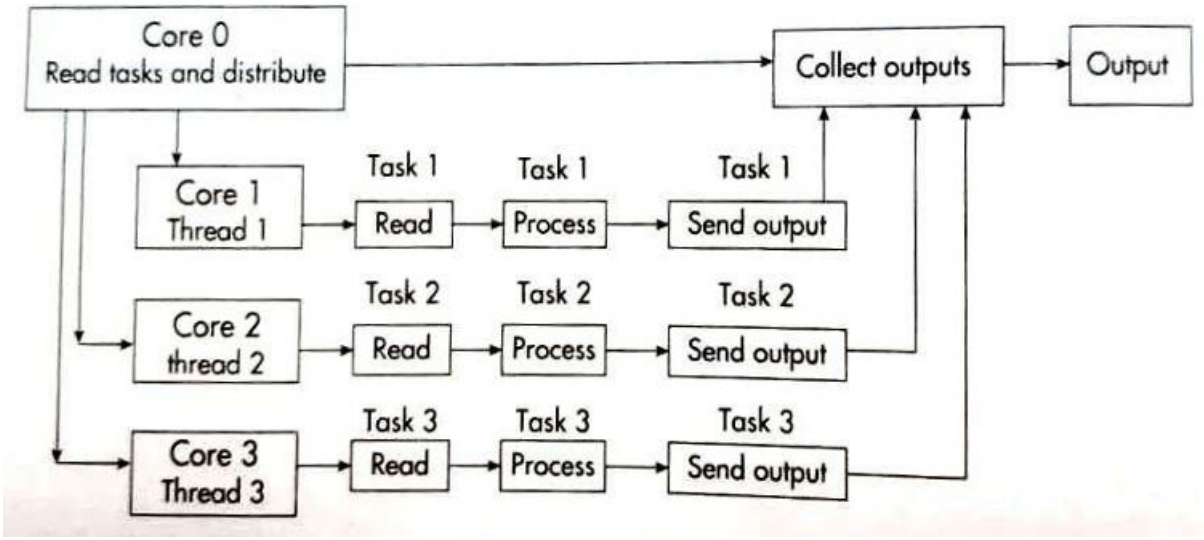


Single Processor: Shares the time of the processor



- Multi Tasking
  - No. of tasks at a time
  - Process based
  - Thread based
    - HT (Hyper-Threading) Technology is a proprietary technology developed by Intel. It allows a single physical CPU core to appear as two logical processors to the operating system.





Process-Based Multitasking	Thread-Based Multitasking
Executes multiple programs.	Executes multiple threads within a program.
Each process has its own address space.	Threads of the same process share the address space.
Processes are comparatively heavyweight.	Threads are lightweight.
Communication between processes is more complex.	Communication between threads is easier through shared memory.
Creating and switching processes generally involves more overhead.	Creating and switching threads generally involves less overhead.

- Process-based multitasking = multiple programs.
  Running a Java program and a music player simultaneously.

- Thread-based multitasking = multiple threads within a program.
  A Java program performing two tasks using two threads.

What is a Multithreading in JAVA?

- Multithreading is a Java feature that allows multiple threads to make progress concurrently within a single program.
- A thread is a lightweight unit of execution. Each thread follows its own path of execution.
- For example, in a music player, different tasks may include:
  - Playing music
  - Updating the user interface
  - Downloading album information
- These tasks can be handled by separate threads.

Important terminology

Term	Simple meaning
Process	A running program
Thread	A path of execution in a program
Multithreading	Running multiple threads in one program
Concurrency	Multiple tasks make progress by taking turns
Parallelism	Multiple tasks execute at the same time
Lightweight process	Another name for a thread

Simple example:
Process: MS Word
Thread: Typing or spell-checking task
Multithreading: Typing and spell-checking together
Concurrency: Two tasks take turns using the CPU.
Parallelism: Two tasks execute simultaneously on different CPU cores.

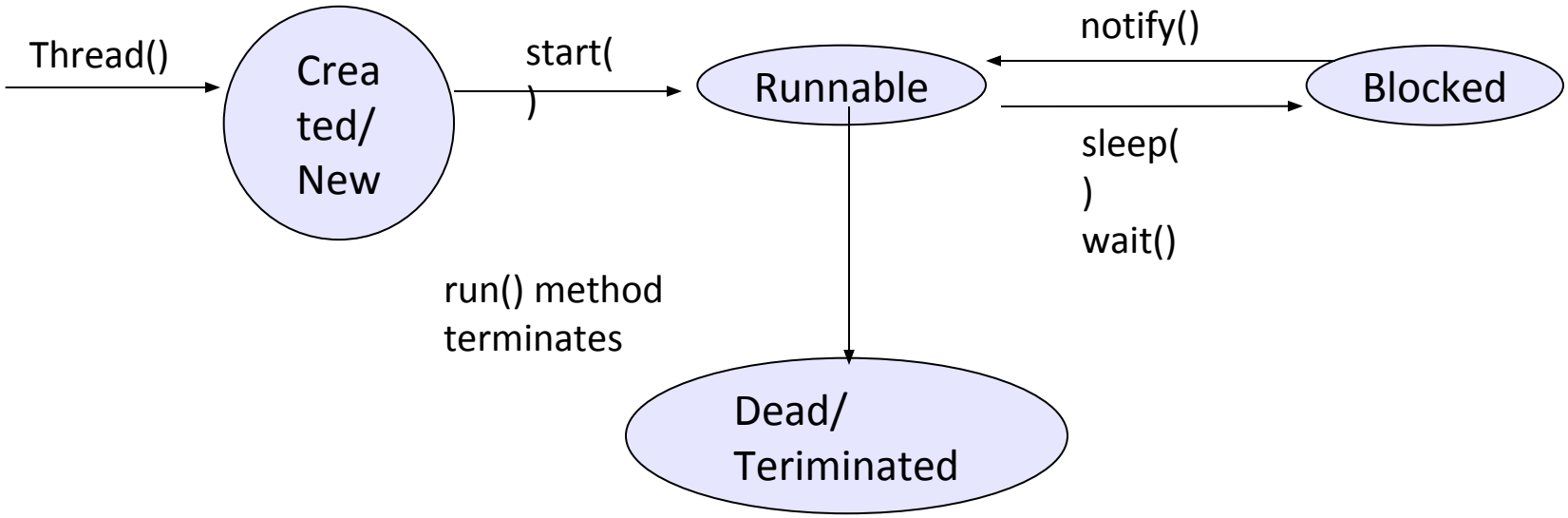
About threads

- So, threads are light-weight processes within a process.
- Threads can be created using :
  - Extending the Thread class
  - Implementing the Runnable Interface
- Advancements in hardware, forcing the development of suitable software for optimal utilization of the processor capacity. Multithread processing is the solution.

- Ex. Main Thread (CurrentThread.java)

Thread

- Lightweight Process (Thread): A thread is a small unit of execution within a process. It shares memory and resources with other threads of the same process, allowing multiple tasks to run concurrently.
- Example: In a browser, one thread may handle the user interface while another downloads the data.
-
-

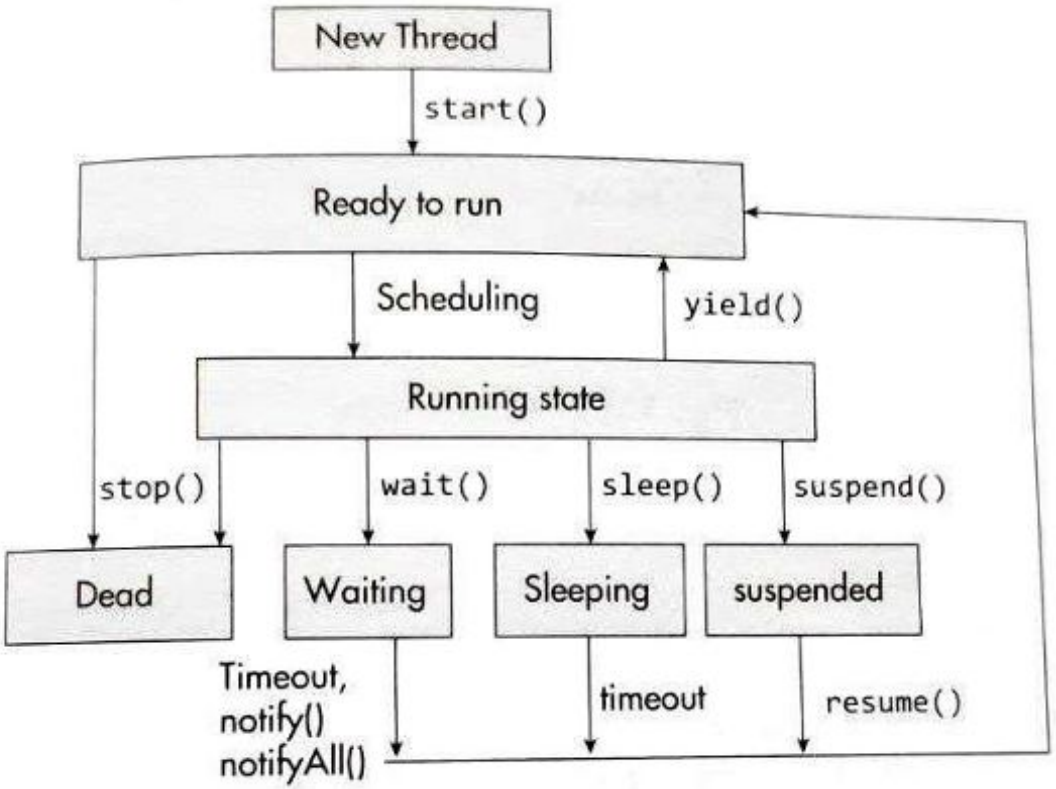


Thread

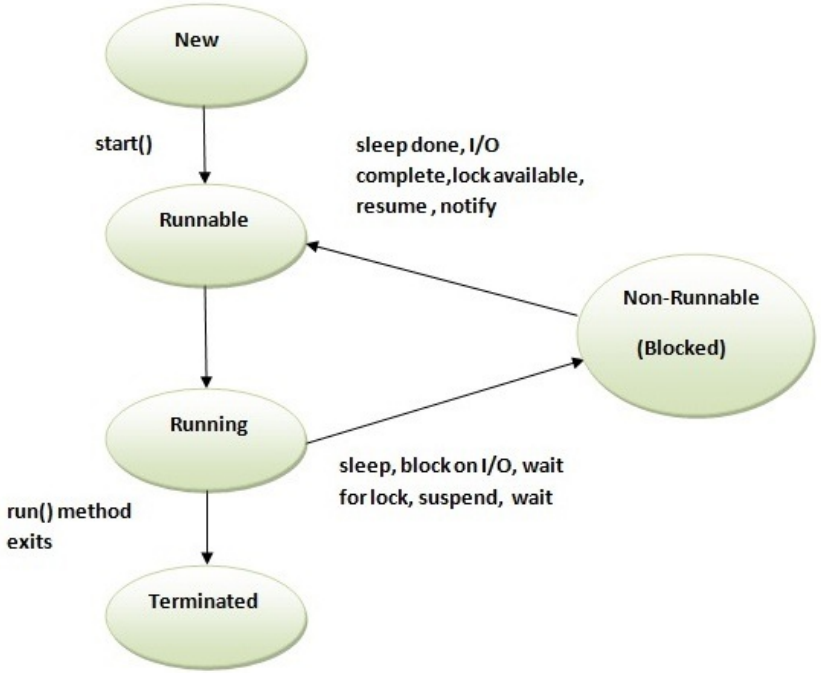
- Light-weight process

  is a means of achieving multitasking- and shares its address space and system resources with other LWPs within the same process.

- Thread State or Life Cycle of a Thread



Thread States



1) **New:** The thread is in new state if you create an instance of Thread class but before the invocation of start() method.

2) **Runnable:** The thread is in runnable state after invocation of start() method, but the thread scheduler has not selected it to be the running thread.

3) Running: The thread is in running state if the thread scheduler has selected it.

4 **) Non-Runnable (Blocked):** This is the state when the thread is still alive, but is currently not eligible to run.

5)Terminated : A thread is in terminated or dead state when its run() method exits.

Thread Creation

- There are two ways to create a thread:
  - By extending Thread class
  - By implementing Runnable interface.

by extending Thread class

- Create a class that extends Thread class. (Ex. MyClass)
- In this class, overwrite(override) run() method.

```
public void run() {
    }
```

  - By default this run() method is recognized and executed by the JVM
  - This is the entry point for thread
- Inside of Main method class, create an object to MyClass.
- Run the thread, for this purpose we have to use start() method.
  - MyClassobject.start();

- Ex: ThreadExtends.java, ThreadExtends2.java, ThreadExtends3.java

Thread Class Constructors

- **Default Constructor**:
  - Thread thread = new Thread();
  - - This creates a new thread with no specific name (by default names starts from Thread-0, Thread-1 and so on) .
- **Thread with Name**:

  Thread thread = new Thread(String name);

  This creates a new thread with a specified name.
  - Ex: Thread_Default_Parameter.java
- **Thread with Runnable Target**:

  Thread thread = new Thread(Runnable target);

  - This creates a new thread that will execute the code defined in the Runnable interface's run method.

  - The name by the compiler as Thread-0, Thread-1, and so on.

  Ex: ThreadClass_Runnable.java

Thread with Runnable Target and Name:

- Thread thread = new Thread(Runnable target, String name);
- This creates a new thread with a specified Runnable target and a name for the thread

Thread with Group, Runnable Target:

- Thread (ThreadGroupgroup, Runnable object)
- It allocates thread with specified group and target.

Thread with Group, Runnable Target, and Name:

- Thread thread = new Thread(ThreadGroup group, Runnable target, String name);
- This creates a new thread that belongs to a specified thread group, with a Runnable target and a name

Ex: ThreadGroupDemo.java

Thread Class Methods

- All threads are objects of class Thread.
- The class Thread contains several methods that control the execution as well as for setting and getting attributes of threads. All are public methods.

Method Signature	Description
String getName()	Retrieves the name of running thread
final void setName(String name)	Set the name to a thread.
void run()	This method is the entry point of the thread. Execution of thread starts from this method.
void start()	This method will start a new thread of execution by calling run() method of Thread.
void sleep(long ms) void sleep(long ms, int ns)	This method suspend the thread for mentioned time duration in argument (sleep time in milli and nano seconds)

Method Signature	Description
boolean isAlive()	This method will check if thread is alive or dead
getState()	Returns the state associated with the thread
int getPriority()/ void setPriority (int value)	Obtains/sets thread priority.
final vold join() final void join(long milliseconds, int nanoseconds)	This method used to queue up a thread in execution. Once called on thread, current thread will wait till calling thread completes its execution
public static Thread currentThread()	returns the current thread instance
static void yield()	he yield() method of the Thread class causes the currently executing thread to temporarily pause and allows other threads of the same or higher priority to get a chance to execute. However, the thread scheduler may choose to execute the same thread again.

long getID()	The getId() method is used to return the thread identifier. The thread ID is a unique positive number which was generated at the time of thread creation.
boolean isInterrupted()	Returns a boolean that indicates whether the specified thread has been interrupted.
String toString()	Returns a string representation of this thread, including the thread's name, priority, and thread group
boolean isDaemon()	is used to check if the current thread is a daemon thread.
ThreadGroup getThreadGroup()	The getThreadGroup() method in Java returns the thread group to which a thread belongs

Implementing Runnable interface

- Create a class that implements Runnable interface. (Ex. MyClass)
- In this class, implement run() method .

  public void run() {

  }

  - By default this run() method is recognized and executed by the thread.
  - This is the entry point for thread
- Inside of Main method class, create an instance of the Thread class and pass your Runnable object to its constructor as a parameter.
- Now Myclass object to **Thread** class
  - Thread t=new Thread(new Myclass());
  - Thread t=new Thread(new MyClass(),"Name");

- Run the thread, for this purpose use start() method.
  - t.start();

Runnable interface method

Method	Description
public void run()	This method takes in no arguments.

Thread Priority

- Thread.MIN_PRIORITY (1) - the lowest priority
- Thread.NORM_PRIORITY (5) - the default priority
- Thread.MAX_PRIORITY (10) - the highest priority
- Thread Priority Setter and Getter Methods
  - public final void setPriority(int newPriority) - specifies new priority to a thread
  - public final int getPriority() – obtain priority value associated with the thread.

Other ways to Create thread

Using method reference:

```
new Thread (Class_Name::Method_Name).start();
```

Using lambda expression:

```
new Thread( () -> Method_Name  or {code to be executed}).start();
```

Using anonymous inner class:

```
new Thread(new Runnable()
        {
          public void run()
          {
                Code to be executed

          }
        }).start();
```

- Using Method Reference
  - A method reference is a short and simple way to refer to an existing method.
  - Syntax:

    ClassName::methodName
- With lambda expression
  - a lambda expression is used to provide a short implementation without creating a separate class or writing an anonymous inner class.
  - The syntax of a lambda expression is:
    - (parameters) -> { statements; }
    - () -> Demo.display()

Creating Multiple Threads – Multi Tasking

- In multi tasking several threads are executed at a time.
- Ex: Thread_Extends_Multiple.java

isAlive()

- boolean isAlive()
  - This method will check if thread is alive or dead

Use of join()

- Join method in Java allows one thread to wait until another thread completes its execution. In simpler words, it means it waits for the other thread to die.
- It has a ***void*** type and throws ***InterruptedException***. Joining threads in Java has three functions namely,

Method	Description
join()	Waits for this thread to die
join(long millis)	Waits at most millis milliseconds for this thread to die
join(long millis, int nanos)	Waits at most millis milliseconds plus nano nanoseconds for this thread to die

Introduction

- The Collection Framework in Java provides classes and interfaces to store and manage a group of objects as a single unit.
- Example: Suppose we want to store the names of five students. Instead of creating five separate variables, we can store all the names in an ArrayList.

```
ArrayList<String> names = new ArrayList<>();

names.add("Ravi");
names.add("Sita");
names.add("Ram");
names.add("Priya");
names.add("Kiran");

System.out.println(names);
```

[Ravi, Sita, Ram, Priya, Kiran]

- Accessing individual elements
  - Even though multiple objects are stored in one collection, we can access each element separately using its index or other suitable methods.

```
System.out.println(names.get(0));
```

- Java's Collection Framework is similar to the Standard Template Library (STL) in C++. Both provide ready-made data structures and operations to manage groups of elements.

Java	C++ STL
ArrayList	vector
LinkedList	list
HashSet	unordered_set
HashMap	unordered_map

- Collections are also called containers
  - Collections are called containers because they hold multiple elements together.
  - For example, an ArrayList acts like a container that stores several student names.
- Elements are stored by their references
  - In Java, collections store references to objects, rather than storing copies of the objects themselves.
- Example:
  - Student s1 = new Student();
  - ArrayList<Student> students = new ArrayList<>();
    students.add(s1);

Introduction

- Generics were introduced in Java 5 (JDK 5) to provide type safety when working with collections.

Allowed:

```
ArrayList list = new ArrayList();
list.add("Ravi");
 list.add(100); // Allowed
```

```
ArrayList<Number> numbers = new
ArrayList<>();

numbers.add(10);     // Integer
numbers.add(25.5);   // Double
numbers.add(100L);   // Long
```

Not Allowed: - from JDK 5

```
ArrayList<String> list = new ArrayList<>();
list.add("Ravi");
list.add(100);       // Compilation error
```

Introduction

- **Interfaces**: A hierarchy of interfaces that defines the operations of collections without specifying their implementation details
- The Iterable interface is at the top of the collection hierarchy, and the Collection interface extends Iterable.
- **Classes** These provide concrete implementation of interfaces.

Introduction

- **Algorithms:** These are methods used to perform operations and manipulate collections and their elements.
- **Maps:** Maps store data in key-value pairs.
  - The first element is called the key, and the second element is called the value associated with that key.
  - Maps are not part of the Collection interface hierarchy, but they are part of the Java Collection Framework.

Purpose of Collection Framework

- The Java Collection Framework provides the following benefits:
  - Uniformity: Provides a common way to work with different types of collections.
  - High Performance: Provides efficient methods to store, access, and manipulate data.
  - Ease of Use: Makes it easier to solve real-world problems by managing groups of objects.

Application of Collection Framework

- It provides different types of collections for storing data.
- It allows us to add or remove elements from a collection.
- It allows us to access each element in a collection.
- It allows us to search for a specific element in a collection.
- It allows us to sort elements in a collection based on a given condition.

Hierarchy of Collection Interfaces/Classes

- The collections framework defines several interfaces.
- At the top of the hierarchy is the *Iterable*, from which interface Collection is derived.
- The interfaces Set, List, and Queue are derived from Collection.
- The interface SortedSet is derived from Set and interface NavigableSet is derived from SortedSet.
- The interface Deque (double-ended queue) is derived from Queue.

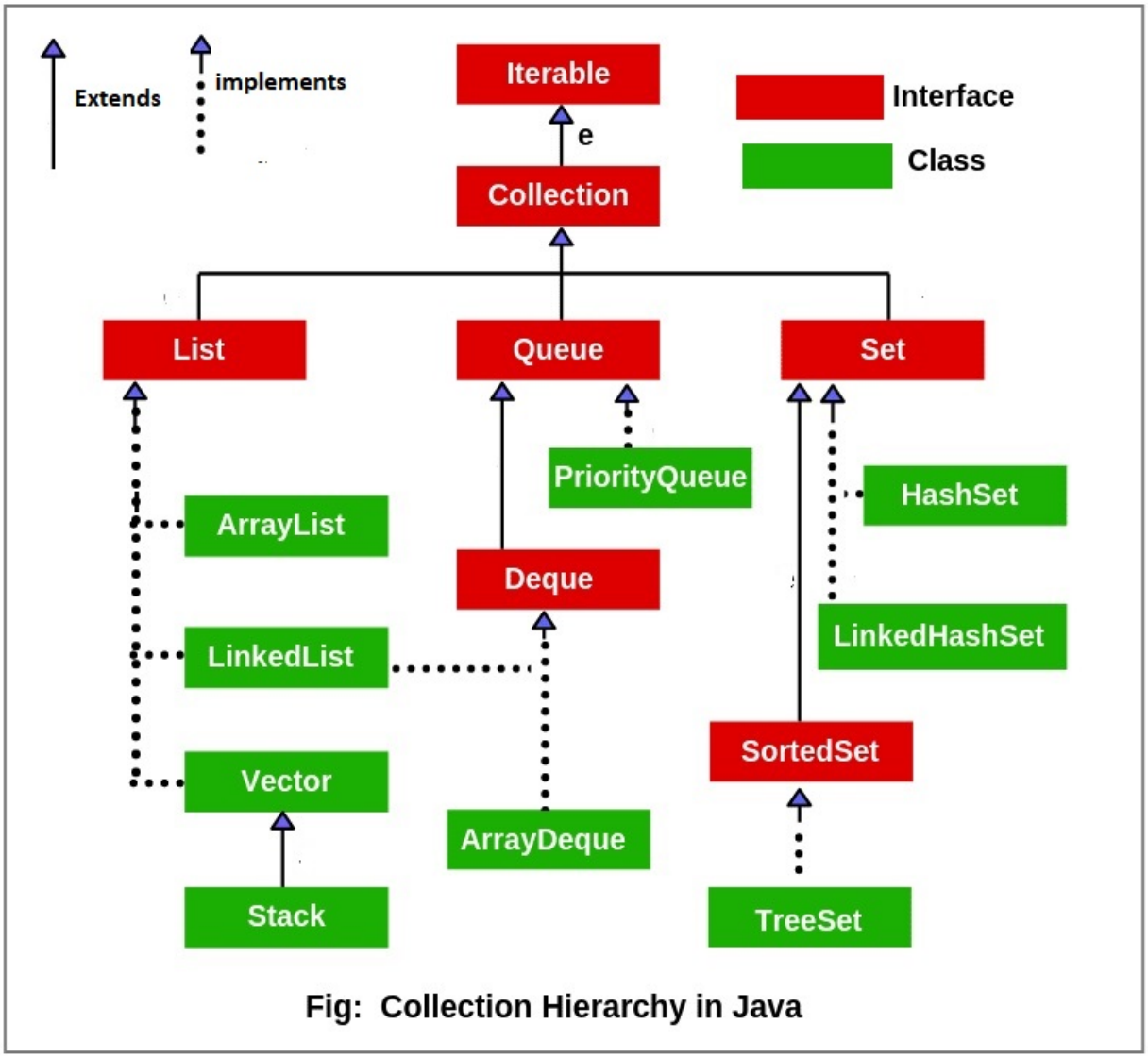


Fig: Collection Hierarchy in Java

Hierarchy of collection interfaces

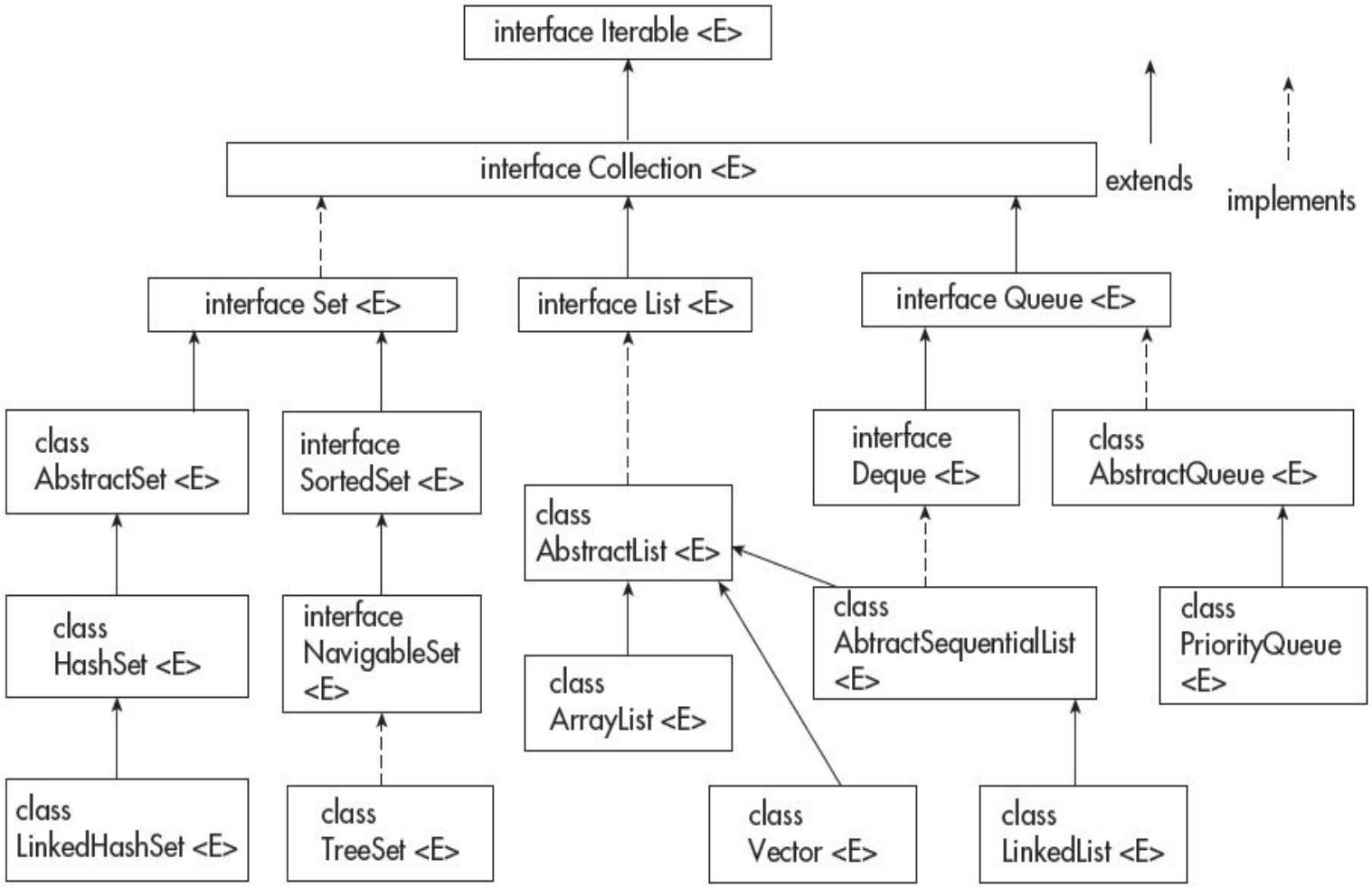


Fig. 16.1 Hierarchy of collection interfaces

Hierarchy of Collection Interfaces/Classes

I. **Iterable Interface**

- The root interface for all collection classes and interfaces.
- Provides an iterator() method to traverse elements in a collection.

II. **Collection Interface**

- Extends Iterable and is the base interface for most collections.
- Defines basic methods like add(), remove(), size(), and clear().

III. Sub-Interfaces of Collection

- **List Interface**
  - An ordered collection (insertion order is preserved) that allows duplicate elements.
  - Elements can be accessed by their index.
  - **Implementations**:
    - **ArrayList**: Resizable array; allows fast random access.
    - **LinkedList**: Doubly linked list; efficient for insertions and deletions.
    - **Vector**: Synchronized, resizable array;
    - **Stack**: LIFO data structure; extends Vector.

III. Sub-Interfaces of Collection

- **Set Interface**
  - A collection that does not allow duplicate elements. Also insertion order is not preserved.
  - **Implementations**:
    - **HashSet**: Uses hash table; does not maintain order.
    - **LinkedHashSet**: Uses a hash table and a linked list. It maintains insertion order

III. Sub-Interfaces of Collection

- **SortedSet interface:**
  - The SortedSet interface in Java is a sub interface of Set that provides additional methods to handle elements in sorted (ascending) order.
  - Elements in a SortedSet must implement the Comparable interface or be ordered by a provided Comparator.
    - **Implementations**:
      - **TreeSet**: Maintains elements in ascending order.

III. Sub-Interfaces of Collection

- **Queue Interface:**
  - Typically represents a collection that holds elements prior to processing.
  - Most queues follow First-In-First-Out (FIFO) order.
  - **Implementations**:
    - **PriorityQueue**: Elements are ordered based on natural ordering or user defined priority.

III. Sub-Interfaces of Collection

- **Deque Interface** (Subinterface of Queue)
  - Deque stands for Double-Ended Queue.
  - It is a subinterface of the Queue interface.
  - It allows elements to be inserted and removed from both the front and the rear ends.
  - **Implementations**:
  - ArrayDeque: Uses a resizable array to implement a double-ended queue.
  - LinkedList: Uses linked nodes and can implement both Deque and Queue.

Hierarchy of maps

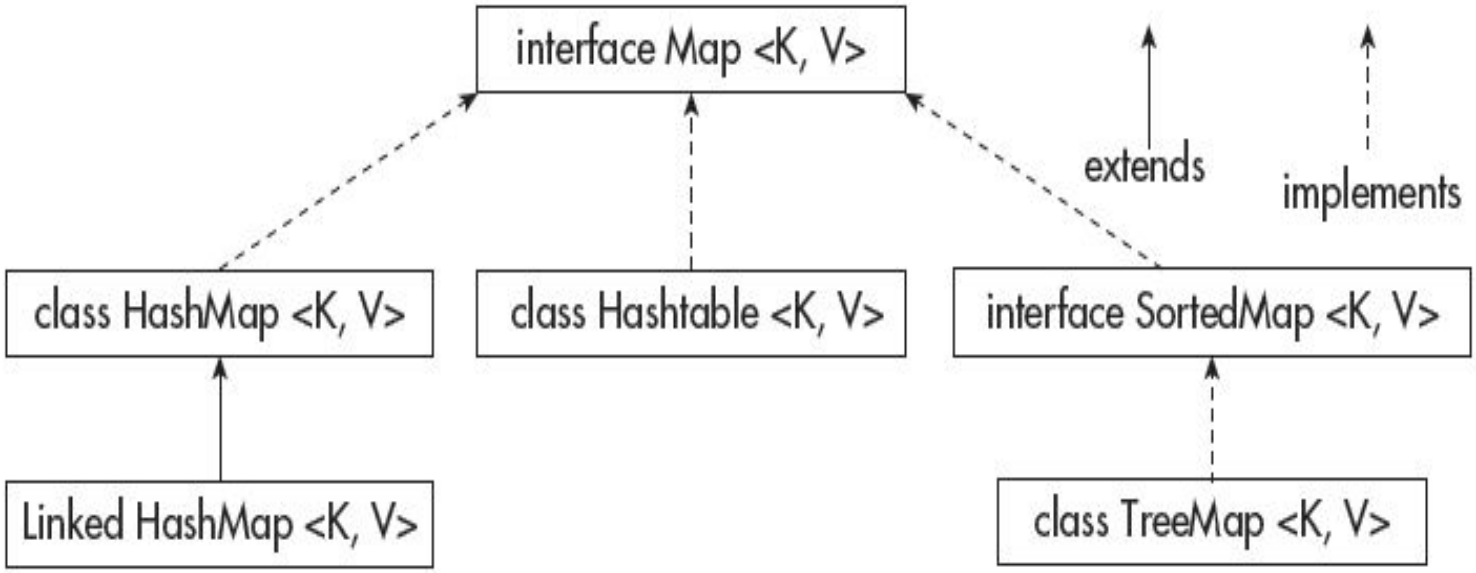


Fig. 16.2 Hierarchy of maps

Hierarchy of Collection Interfaces/Classes

- The Map interface in Java is part of the Java Collections Framework and provides a way to store data in key-value pairs.
- Unlike other collection interfaces (Collection, Set, List), Map does not inherit from Collection because it is designed specifically for mapping unique keys to values rather than holding individual elements.

Implementations:

HashMap	Uses a hash table. Allows one null key and multiple null values. Does not guarantee order.
LinkedHashMap	Maintains insertion order by default. Allows one null key and multiple null values.
Hashtable	Synchronized (thread-safe) hash table. Does not allow null keys or null values.
SortedMap	An interface that maintains entries sorted by their keys, usually in ascending order.
TreeMap	Implements SortedMap using a Red-Black tree. Maintains keys in sorted order.

Summary of Core Classes and Interfaces

Interface	Description	Implementation Classes
List	Ordered collection, allows duplicates	ArrayList, LinkedList, Vector, Stack
Set	Unordered collection, no duplicates	HashSet, LinkedHashSet
SortedSet	Ordered collection, no duplicates	TreeSet
Queue	FIFO ordering, holds elements for processing	LinkedList, PriorityQueue,
Deque	Double-ended queue	ArrayDeque, LinkedList
Map	Key-value pairs, unique keys	HashMap, LinkedHashMap, TreeMap, Hashtable

Hierarchy of Collection Interfaces/Classes

- Java **Iterator** Interface of java collections allows us
  - to access elements of the collection
  - and is used to iterate over the elements in the collection (Ex: List or Set).
- **Iterator** is a universal iterator as it can be applied to any Collection object.
- We can traverse only in the **forward** direction using iterator.
- Almost all collections support iterators except the maps. In maps, the elements are accessed through keys.

Iterator interface

- Iterator interface is used in Java Collections framework in the place of Enumeration.
  - Enumeration works only on legacy classes such as Vector and Hashtable.
  - Removing element the given collection during iterating over the elements is not possible in enumeration.
  - Only forward direction is possible in an Enumeration.

Iterator interface

Method	Purpose
boolean hasNext()	Checks if more elements are present or not
E next()	Retrieves the next element
void remove()	Removes the last element returned by the next() method from the collection.

ListIterator Interface

- Primarily used in list implementations like ArrayList and LinkedList, where both forward and backward traversal is needed, or where elements might need to be replaced or added during iteration.
- Additional Methods:
- **boolean hasPrevious()**: Checks if there is an element before the current element in the list.
- **E previous()**: Returns the previous element in the list.
- **int nextIndex()**: Returns the index of the element that would be returned by next().
- **int previousIndex()**: Returns the index of the element that would be returned by previous().
- **void set(E e)**: Replaces the last element returned by next() .
- **void add(E e)**: Inserts the specified element into the list at the iterator's current position.

Methods Declared in Collection Interface

- The Collection interface represents a unified architecture for performing different operations on a group of objects such as searching, sorting, insertion, manipulation, and deletion.
- The methods defined in this interface are inherited by all sub-interfaces.
- These methods are implemented by the classes that deal with the respective collections.

Methods Defined in Collection Interface

Method	Description
boolean add(Object obj)	Adds obj to the invoking collection. Returns true if obj was added to the collection. Returns false if obj is already a member of the collection, or if the collection does not allow duplicates.
boolean addAll(Collection c)	Adds all the elements of c to the invoking collection. Returns true if the operation succeeds (i.e., the elements were added). Otherwise, returns false.
void clear( )	Removes all elements from the invoking collection.
boolean contains(Object obj)	Returns true if obj is an element of the invoking collection. Otherwise, returns false.
boolean containsAll(Collection c)	Returns true if the invoking collection contains all elements of **c**. Otherwise, returns false.

Method	Description
boolean equals(Object obj)	Returns true if the invoking collection and obj are equal. Otherwise, returns false.
int hashCode( )	Returns the hash code for the invoking collection.
boolean isEmpty( )	Returns true if the invoking collection is empty. Otherwise, returns false.
Iterator iterator( )	Returns an iterator for the invoking collection.
boolean remove(Object obj)	Removes one instance of obj from the invoking collection. Returns true if the element was removed. Otherwise, returns false.
boolean removeAll(Collection c)	Removes all elements of c from the invoking collection. Returns true if the collection changed (i.e., elements were removed). Otherwise, returns false.

Method	Description
boolean retainAll(Collection c)	Removes all elements from the invoking collection except those in c. Returns true if the collection changed (i.e., elements were removed). Otherwise, returns false.
int size( )	Returns the number of elements held in the invoking collection
Object[ ] toArray( )	Returns an array that contains all the elements stored in the invoking collection. The array elements are copies of the collection elements.
<T>[ ] toArray (T array[ ])	Returns an array containing only those collection elements whose type matches that of array.

Exceptions Thrown by Methods

1. **UnsupportedOperationException:** If the invoking collection cannot be supported for a given operation, several of the above methods may throw UnsupportedOperationException.
2. **ClassCastException:** If addition of an object of type different from that of the collection is attempted, it may throw ClassCastException.
3. **NullPointerException:** If the Collection interface does not allow null object, and hence, if addition of null object is attempted NullPointerException is thrown.
4. **IllegalStateException:** If the collection is of fixed length and is full, an attempt to add another element will result in IllegalStateException.
5. **IllegalArgumentException:** Use of an invalid argument will result in IllegalArgumentException.

ArrayList Class

- The ArrayList class implements a dynamic array whose size can grow or shrink automatically.
- It is useful when the number of elements to be stored is not known in advance.
- The ArrayList class extends AbstractList and implements the List, RandomAccess, Cloneable, and Serializable interfaces.

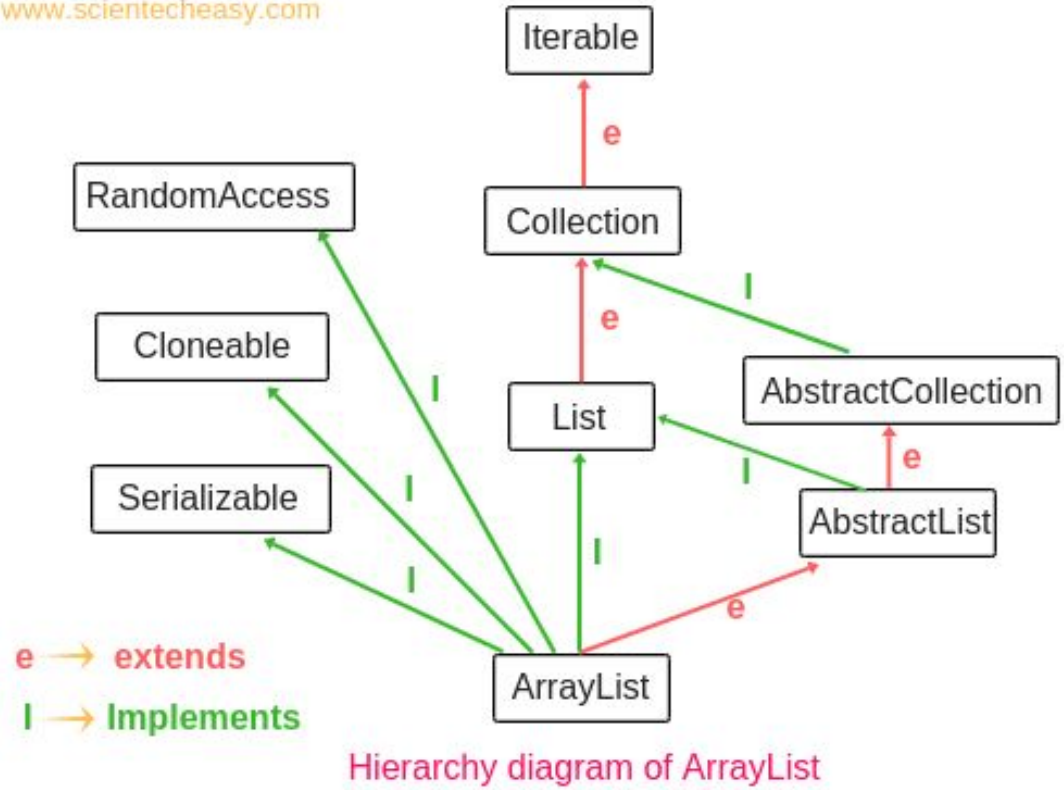
- It is declared as follows:

public class ArrayList<E> extends AbstractList<E>

implements List<E>, RandomAccess, Cloneable, Serializable

- E represents the type of elements stored in the ArrayList. For example, ArrayList<Integer> stores integers, while ArrayList<String> stores strings.

www.scientecheasy.com



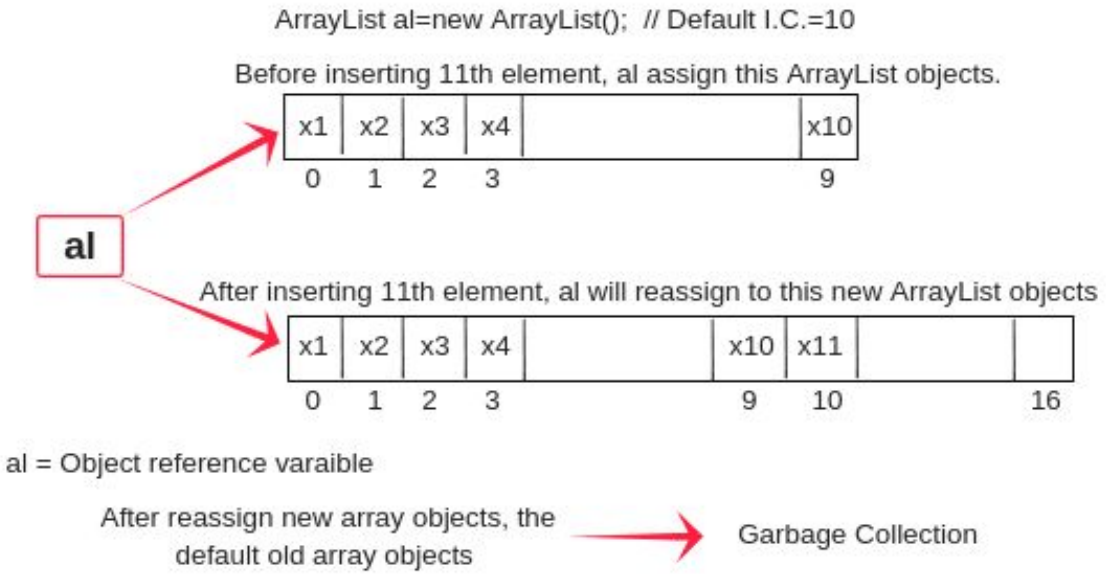
Hierarchy diagram of ArrayList

Constructors of class ArrayList

1. ArrayList(): This constructs an empty list with initial default capacity of 10.
2. ArrayList(int Capacity) This constructor method constructs an empty list having a specified initial capacity.
3. ArrayList: (Collection *Collection*)
   - It constructs a list with elements of a specified collection.
   - The elements are kept in the order in which they are delivered to ArrayList instance.

ArrayList is the best choice for efficient retrieval purpose (random access is possible), not for insertion or deletion because several shifting operations are required

- ArrayList al = new ArrayList();



- Once ArrayList is reached its maximum capacity, the ArrayList class automatically creates a new array with a larger capacity.
- **New capacity** = (current capacity*3/2) + 1 = 10*3/2 + 1 = 16
- After creating a new array list with a larger capacity, all the existing elements are copied into the new array list and then adds the new element into it.

Methods of Class ArrayList

1. Adding Elements

- **add(E e)**: Adds the specified element to the end of the list.
- **add(int index, E element)**: Inserts the specified element at the specified position in the list.
- **addAll(Collection c)** – adds the collection at the end

2. Accessing Elements

- **get(int index)**: Returns the element at the specified position in the list.

3. Updating Elements

- **set(int index, E element)**: Replaces the element at the specified position with the specified element.

4. Removing Elements

- **remove(int index)**: Removes the element at the specified position in the list.
- **remove(Object o)**: Removes the first occurrence of the specified element from the list.
- **removeAll(Collection)** – removes all the elements specified in the collection
- **retailnAll(Collection c)** – only the specified elements in the collection will be retained

5. Searching and Checking Existence

- **contains(Object o)**: Returns true if the list contains the specified element.
- **indexOf(Object o)**: Returns the index of the first occurrence of the specified element, or -1 if not found.
- **lastIndexOf(Object o)**: Returns the index of the last occurrence of the specified element, or -1 if not found.

6. Size and Capacity

- **size()**: Returns the number of elements in the list.
- **isEmpty()**: Returns true if the list contains no elements.
- **ensureCapacity(int minCapacity)**: Increases the capacity of the ArrayList instance.

7. Iterating and Traversing

- **iterator()**: Returns an iterator over the elements in the list.
- **listIterator()**: Returns a list iterator for traversing the list in either direction

8. Other Utility Methods

- **clear()**: Removes all elements from the list.
- **toArray()**: Converts the list to an array.
- **subList(int fromIndex, int toIndex)**: Returns a view of a portion of the list.

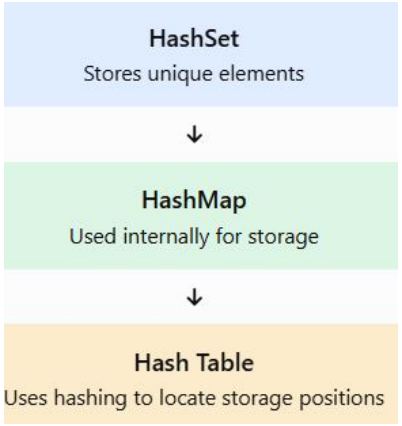
Set

- The Set interface extends Collection interface so that sets can contain a collection of objects, which are unique.

- The Set interface does not contain any of the methods its own.

HashSet Class

- A HashSet is a class in Java that is used to store unique elements without duplicates.
  - It uses a HashMap internally to store elements.
  - It implements the Set interface.
  - It does not maintain insertion order.
  - It allows at most one null element.
  - It is part of the java.util package.



Syntax:

public class HashSet<e> extends AbstractSet<e> implements Set<e>, Cloneable, Serializable

Key points of HashSet class are:

- HashSet stores the elements by using a mechanism called hashing.
- HashSet contains unique elements only(no duplicates).
- HashSet allows null value.
- HashSet class is non-synchronized (not thread safe).
- HashSet doesn't maintain the insertion order. Here, elements are inserted on the basis of their hashcode.

Constructors

- HashSet h = new HashSet(); is the default constructor. Default initial capacity is 16, load factor is 0.75.
- HashSet h = new HashSet(int initialCapacity); is a constructor with a given initial capacity. The load factor is 0.75.
- HashSet h = new HashSet(int initialCapacity, float loadFactor); is a constructor with a given initial capacity and load factor.
- HashSet h = new HashSet(Collection C); is a constructor that adds elements from another collection.

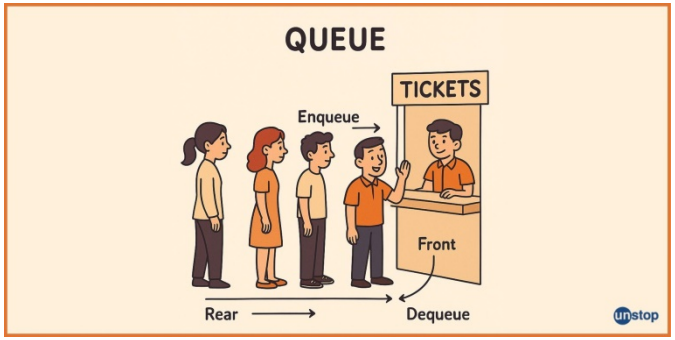
- The new capacity after resizing is usually: **New Capacity=Old Capacity×2;**

Methods:

- **boolean add(E e):** adds an element to HashSetif it doesn't exist, if it already exists, the method returns false .
- **void clear():**removes all elements from the set.
- **boolean contains(Object o):** Returns true if the given element is present in the set.
- **boolean remove(Object o):** Removes the given element from the set, if any.
- **Iterator iterator():** returns an iterator for the elements of the set.
- **boolean isEmpty():** returns true if the set has no elements.
- **Object clone():** performs a shallow clone HashSet.

Java PriorityQueue

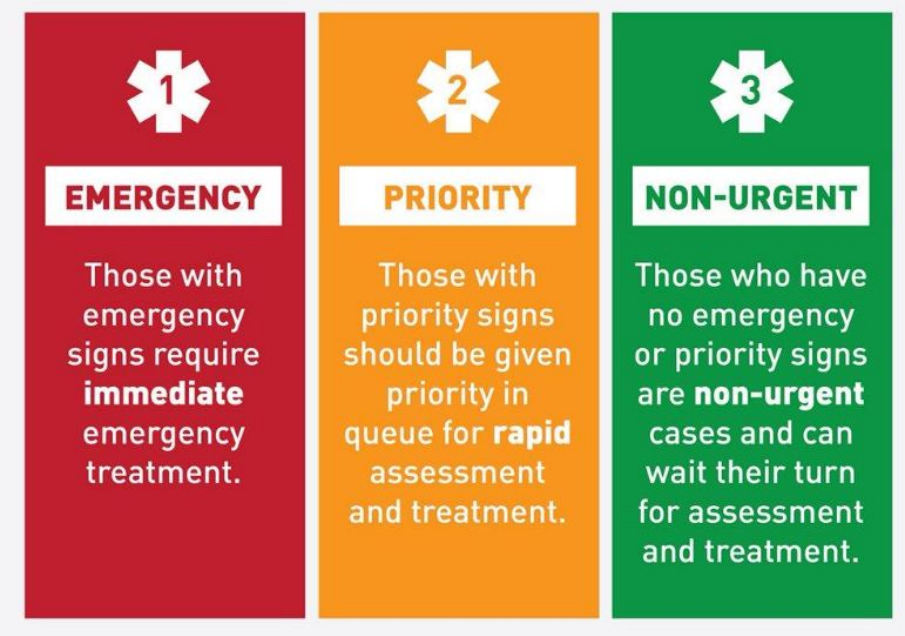
- A Queue is a data structure that generally follows FIFO.
- FIFO means First In, First Out.
- Suppose four students enter a queue:
  - Ravi
  - Sita
  - Kumar
  - Rani



- Who will get the ticket first?
  - Answer: Ravi, because he entered first.
  - This is called FIFO.

Why do we need PriorityQueue?

- Now consider a hospital emergency department.



Three patients arrive at a hospital:

Patient	Arrival order	Medical priority
Ravi	1	3
Sita	2	1
Kumar	3	2

Here, assume priority 1 means the highest priority.
Who should receive treatment first?
Sita should receive treatment first, even though Ravi arrived before her.
Why?
Because Sita has the highest priority.
This is the basic idea behind a PriorityQueue.

PriorityQueue

- A PriorityQueue is a collection in Java in which elements are processed based on their priority rather than their insertion order.
- Java provides a class called PriorityQueue in the java.util package.
- It implements the Queue interface.

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
```

  - PriorityQueue – Class name.
  - Integer – Type of elements.
  - pq – Reference variable.
  - new PriorityQueue<>() – Creates a PriorityQueue object.

```
import java.util.*;

public class PQDemo {
   public static void main(String[] args) {

      PriorityQueue<Integer> pq = new PriorityQueue<>();

      pq.add(40);
      pq.add(10);
      pq.add(30);
      pq.add(20);

      System.out.println(pq);
   }
}
```

[10, 20, 30, 40]

- By default, PriorityQueue follows natural ordering for Integer elements.
- Natural ordering of integers means ascending order.
- Therefore, the smallest element gets the highest priority.

Understand peek() and poll()

Method	Function
peek()	Retrieves the highest-priority element without removing it
poll()	Retrieves and removes the highest-priority element

```
import java.util.*;

public class PQDemo {
   public static void main(String[] args) {

      PriorityQueue<Integer> pq = new PriorityQueue<>();

      pq.add(40);
      pq.add(10);
      pq.add(30);
      pq.add(20);

      System.out.println(pq.peek());

      System.out.println(pq.poll());

      System.out.println(pq);
   }
}
```

```
10
10
[20, 40, 30]
```

How to retrieve all elements in priority order?

```
import java.util.*;

public class PQDemo {
   public static void main(String[] args) {

      PriorityQueue<Integer> pq =
            new PriorityQueue<>();

      pq.add(40);
      pq.add(10);
      pq.add(30);
      pq.add(20);

      while (!pq.isEmpty()) {
         System.out.println(pq.poll());
      }
   }
}
```

Class Declaration

The class is declared as

public class PriorityQueue <E>

extends AbstractQueue <E>

implements Queue, serializable

Constructors of Class PriorityQueue

PriorityQueue<E> pq = new PriorityQueue<>(); -

Creates an empty priority queue with the default initial capacity (11) and natural ordering.

PriorityQueue<E> pq = new PriorityQueue<>(int capacity);

Creates a priority queue with the specified initial capacity and natural ordering.

PriorityQueue<E> pq = new PriorityQueue<>(Comparator<E> comparator);

Creates a priority queue with a custom comparator for ordering elements.

- **PriorityQueue<E> pq = new PriorityQueue<>(int initialCapacity, Comparator<E> comparator);**

  Creates a priority queue with both a specified capacity and custom comparator.

- **PriorityQueue<E> pq = new PriorityQueue<>(Collection<E> c);**

  Creates a priority queue containing the elements of the specified collection, using natural ordering.

Methods

- **boolean add(E e);** - Inserts the specified element into the queue. Throws an exception if the queue is full.
- **boolean offer(E e);** - Inserts the specified element into the queue. Returns false if it cannot be added (unlike add, it doesn't throw an exception).
- **E poll();** - Retrieves and removes the head of the queue. Returns null if the queue is empty.
- **E peek();** - Retrieves the head of the queue without removing it. Returns null if the queue is empty.

- **boolean remove(Object o);** - Removes a single instance of the specified element from the queue, if it exists.
- **boolean contains(Object o);** - Returns true if the queue contains the specified element.
- **void clear();** - Removes all elements from the queue.
- **int size();** - Returns the number of elements in the queue.
- **boolean isEmpty();** - Returns true if the queue contains no elements.
- Iterator iterator() – returns iterator instance.
- **Object[] toArray();** - Returns an array containing all elements in the queue.

What are the characteristics of PriorityQueue?

- It implements the Queue interface.
- It follows priority-based processing.
- Duplicate elements are allowed.
- Null elements are not allowed.
- By default, it follows natural ordering.
- It is not synchronized.

What if we want descending order?

- So far, we have seen that the default PriorityQueue retrieves the smallest element first.
- But suppose we want the largest element first.
- For example:
  - Inserted elements: 10, 40, 20, 30
  - Expected retrieval order: 40, 30, 20, 10
  - How can we achieve this?
    - We use Comparator.
    - Comparator allows us to define our own priority ordering.

```
import java.util.*;

public class PQDemo {
   public static void main(String[] args) {

      PriorityQueue<Integer> pq =
         new PriorityQueue<>(Collections.reverseOrder());

      pq.add(10);
      pq.add(40);
      pq.add(20);
      pq.add(30);

      while (!pq.isEmpty()) {
         System.out.println(pq.poll());
      }
   }
}
```

```
40
30
20
10
```

Collections.reverseOrder() tells Java to arrange the priority in reverse order.

PriorityQueue	Output using poll()
new PriorityQueue<>()	Smallest number first
new PriorityQueue<>(Collections.reverseOrder())	Largest number first

How do we change the ordering?
Using Comparator.

How does Comparator work?



The important abstract method is (for integers):

```
public interface Comparator<T>
{
        int compare(T a, T b);
}
```

Here: It compares two objects.

One important point

- compare(a, b) → Ascending order.
- compare(b, a) → Descending order.

Return value	Meaning
Negative value	a gets higher priority than b
Zero	Both have equal ordering priority
Positive value	b gets higher priority than a

Ex Comparator with Student objects

Suppose we have three students:

Student	Marks
Ravi	85
Sita	95
Ram	75

Our requirement is to give priority to the student who has the highest marks.

Create a Student class

```
class Student {
   String name;
   int marks;

   Student(String name, int marks) {
      this.name = name;
      this.marks = marks;
   }
}
```

```
Student s1 = new Student("Ravi", 85);
Student s2 = new Student("Sita", 95);
Student s3 = new Student("Ram", 75);
```

Create our own Comparator

```
class MyComparator implements Comparator<Student> {

   public int compare(Student s1, Student s2) {
      return Integer.compare(s2.marks, s1.marks);
   }
}
```

- What does Comparator<Student> mean?
  - It means that our Comparator compares Student objects.
- What does this statement mean?
  - return Integer.compare(s2.marks, s1.marks)
  - Integer.compare(95, 85);
  - The student with 95 marks gets higher priority.


Student objects
Ravi (85), Sita (95), Ram (75)
Comparator
Compares students based on marks
PriorityQueue
Highest marks get priority
Retrieval using poll()
95 → 85 → 75

- For Strings
  - compareTo() compares Strings lexicographically (dictionary-style character order).
  - a.compareTo(b) → Natural String order.
  - b.compareTo(a) → Reverse String order.
- String comparison is case-sensitive by default.

Call Center App

First call
Bob (Urgency 3) Remaining queue: Alice → Danny → Richard
Second call
Alice (Urgency 2) Remaining queue: Danny → Richard
Third call
Danny (Urgency 2) Remaining queue: Richard
Fourth call
Richard (Urgency 1)

Our priority insertion logic is correct.
Urgency 3 → Highest priority.
Urgency 2 → Medium priority.
Urgency 1 → Lowest priority.
Same urgency → First Come, First Served (FCFS).

Customer	Urgency	Arrival order
Alice	2	1
Bob	3	2
Danny	2	3
Richard	1	4